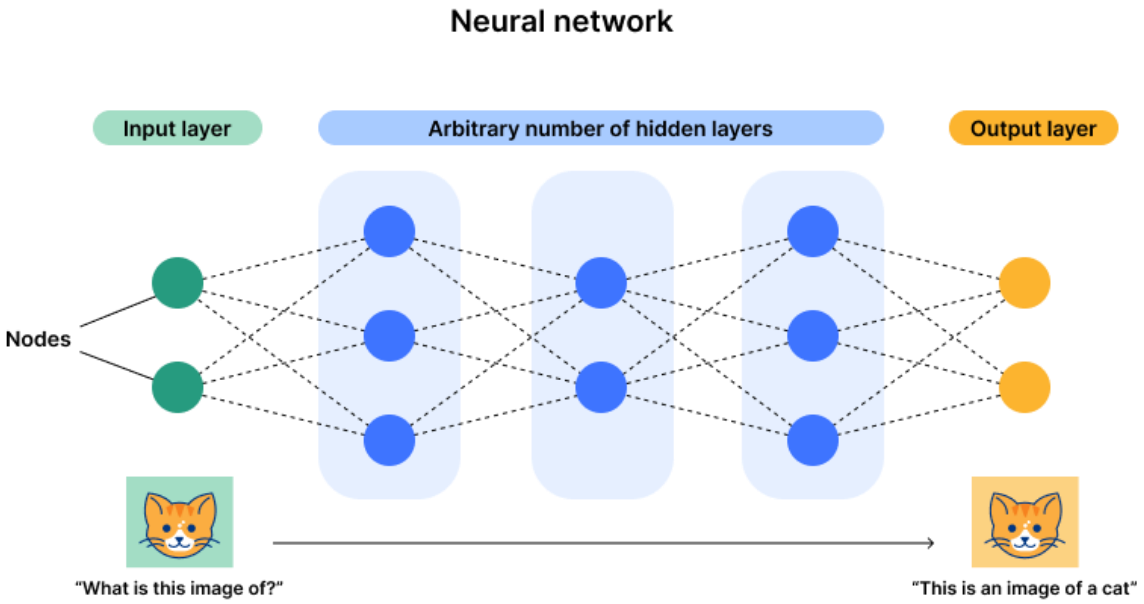
Unit- III

Neural Networks: The perceptron- learning rate, bias, perceptron learning algorithm, linear separability-Exclusive-OR function, backpropagation algorithm, initialization of weights, MLP in practice, deriving back propagation, radial basis functions. An illustrative example: face recognition.

Neural Network

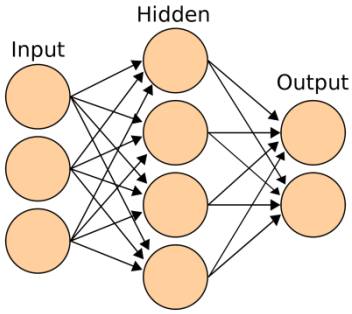
A neural network, or artificial neural network, is a type of computing architecture that is based on a model of how a human brain functions — hence the name "neural." Neural networks are made up of a collection of **processing units called "nodes**." These nodes pass data to each other, just like how in a brain, neurons pass electrical impulses to each other.



How do neural networks work:

Neural networks are composed of a collection of nodes. The nodes are spread out across at least three layers. The three layers are:

- An input layer
- A "hidden" layer
- An output layer



These three layers are the minimum. Neural networks can have more than one hidden layer, in addition to the input layer and output layer.

No matter which layer it is part of, each node performs some sort of processing task or function on whatever input it receives from the previous node (or from the input layer). Essentially, each node contains a mathematical formula, with each variable within the formula weighted differently. If the output of applying that mathematical formula to the input exceeds a certain threshold, the node passes data to the next layer in the neural network. If the output is below the threshold, no data is passed to the next layer.

Weights

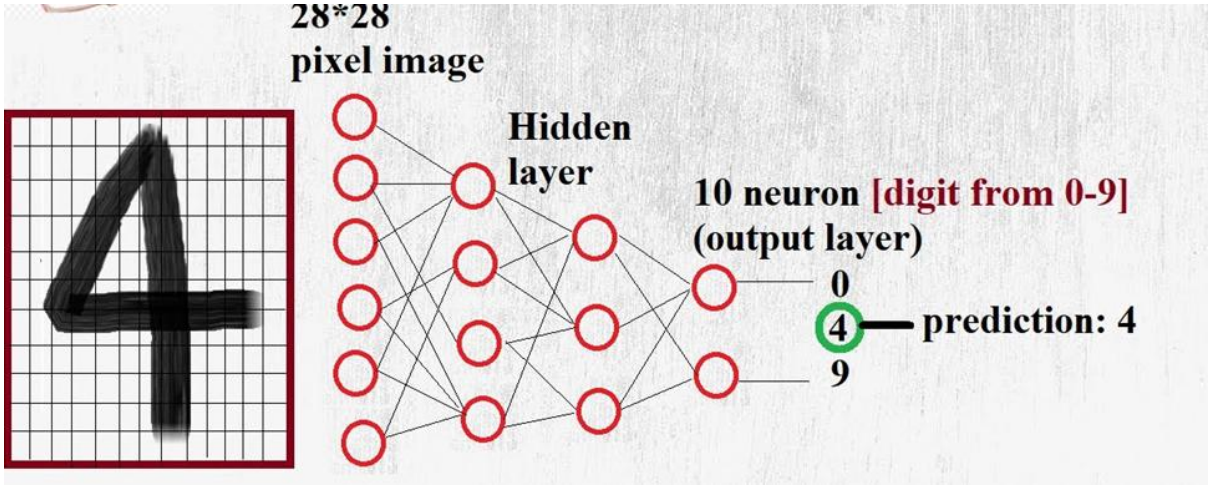
Weights are numerical values associated with the connections between neurons. They determine the strength of these connections and, in turn, the influence that one neuron's output has on another neuron's input. **Think of weights as the coefficients that adjust the impact of incoming data.** They can increase or decrease the importance of specific information.

During the training phase of a neural network, these weights are adjusted iteratively to minimize the difference between the network's predictions and the actual outcomes.

Let's consider a practical example to illustrate the role of weights. Suppose you're building a neural network to recognize handwritten digits. Each pixel in an image of a digit can be considered an input to the network. The weights associated with each pixel determine how much importance the network places on that pixel when making a decision about which digit is represented in the image.

As the network learns from a dataset of labeled digits, it adjusts these weights to give more significance to pixels that are highly correlated with the correct digit and less significance to pixels that are less relevant. Over time, the network learns to recognize patterns in the data and make accurate predictions.

In essence, weights are the neural network's way of learning from data. They capture the relationships between input features and the target output, allowing the network to generalize and make predictions on new, unseen data.



II. Bias

While weights determine the **strength of connections between neurons**, biases provide a critical **additional layer of flexibility** to neural networks. **Biases are essentially constants associated with each neuron**. Unlike weights, biases are not connected to specific inputs but are added to the neuron's output.

Biases serve as a form of offset or threshold, allowing neurons to activate even when the weighted sum of their inputs is not sufficient on its own. They introduce a level of adaptability that ensures the network can learn and make predictions effectively.

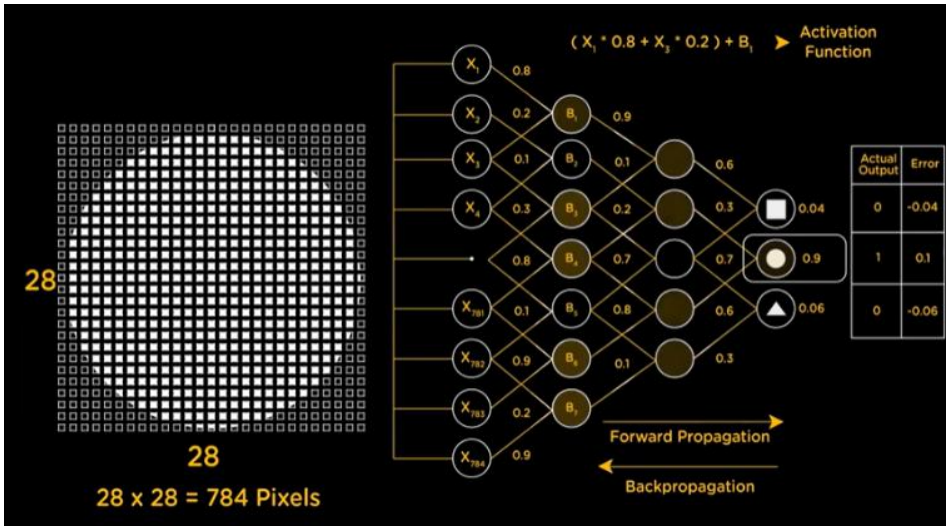
To understand the role of biases, consider a simple example. Imagine a neuron that processes the brightness of an image pixel. Without a bias, this neuron might only activate when the pixel's brightness is exactly at a certain threshold. However, by introducing a bias, you allow the neuron to activate even when the brightness is slightly below or above the threshold.

This flexibility is crucial because real-world data is rarely perfectly aligned with specific thresholds. Biases enable neurons to activate in response to various input conditions, making neural networks more robust and capable of handling complex patterns.

During training, biases are also adjusted to optimize the network's performance. They can be thought of as fine-tuning parameters that help the network fit the data better.

III. The Learning Process: Forward and Backward Propagation

Now that we understand the roles of weights and biases, let's explore how they come into play during the learning process of a neural network.



A. Forward Propagation

Forward propagation is the initial phase of processing input data through the neural network to produce an output or prediction. Here's how it works:

1. **Input Layer:** The input data is fed into the neural network's input layer.

2. **Weighted Sum:** Each neuron in the subsequent layers calculates a weighted sum of the inputs it receives, where the weights are the adjustable parameters.

3. **Adding Biases:** To this weighted sum, the bias associated with each neuron is added. This introduces an offset or threshold for activation.

4. **Activation Function:** The result of the weighted sum plus bias is passed through an activation function. **This function determines whether the neuron should activate or remain inactive based on the calculated value.**

5. **Propagation:** The output of one layer becomes the input for the next layer, and the process repeats until the final layer produces the network's prediction.

B. Backward Propagation

Once the network has made a prediction, it's essential to evaluate how accurate that prediction is and make adjustments to improve future predictions. This is where backward propagation comes into play:

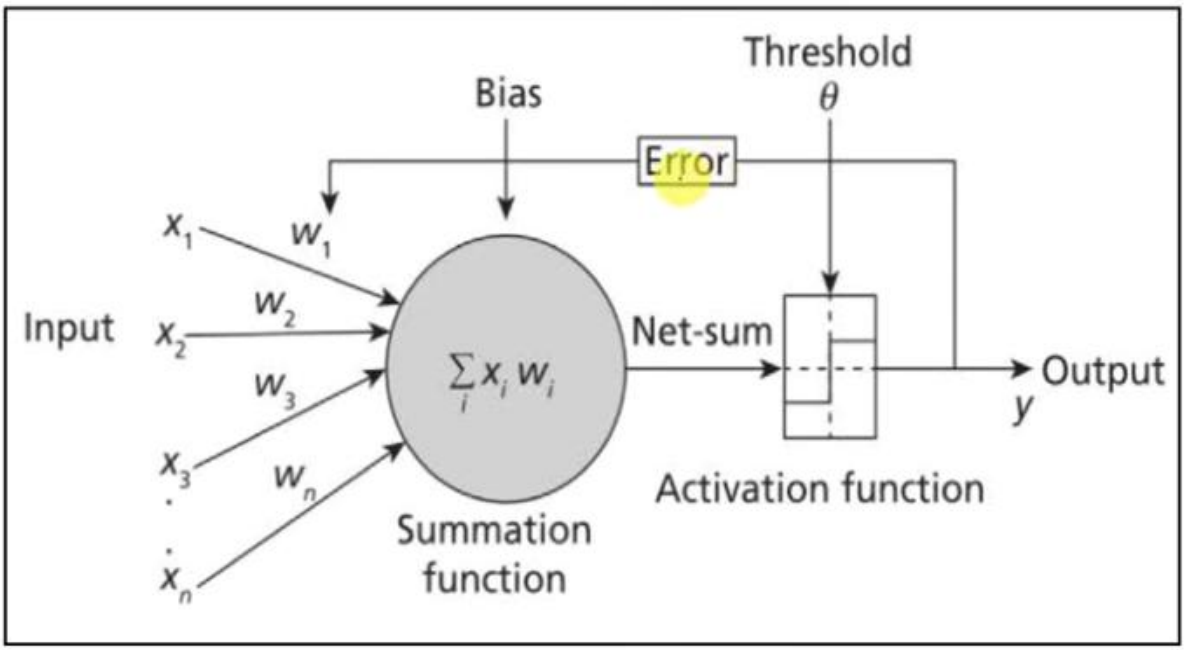
1. **Error Calculation:** The prediction made by the network is compared to the actual target or ground truth. The resulting error, often quantified as a loss or cost, measures the disparity between prediction and reality.
2. **Gradient Descent:** During training, weights and biases are adjusted through an optimization process, often using a technique called gradient descent. The network calculates the gradient of the error (the difference between its predictions and the true values) with respect to the weights and biases. This gradient points in the direction of the steepest decrease in error.

3. **Bias Updates:** The network uses this gradient information to update the weights and biases throughout the network. The goal is to find the values that minimize the error.

4. **Iterative Process:** This process of forward and backward propagation is repeated iteratively on batches of training data. With each iteration, the network's weights and biases get closer to values that minimize the error.

The Perceptron

- A perceptron is the simplest form of an artificial neuron
- A perceptron takes multiple inputs, multiplies each by a weight, adds a bias, and passes the result through an activation function to produce an output.



How Does a Perceptron Work?

The perceptron works by receiving inputs, which could be feature values from a dataset, and combining them with a set of weights. These weights represent the strength of the connection between the inputs and the neuron. The perceptron's formula can be expressed as:

output = f(w1*x1 + w2*x2 + ... + wn*xn + b)

where:

- **w1, w2, ..., wn** are the weights,
- **x1, x2, ..., xn** are the input signals,
- **b** is the bias, which allows the activation function to be shifted to the left or right, to better fit the data,
- **f** is the activation function, typically a step function that outputs either 0 or 1.

The perceptron's decision-making process is binary. If the sum of the weighted inputs plus the bias is greater than zero, the perceptron outputs a 1; otherwise, it outputs a 0. This binary step function is what allows the perceptron to classify input data.

- A perceptron can only solve linearly separable problems like AND, OR, and NOT gates, but fails on non-linear problems like XOR.

Perceptron Example

Imagine a perceptron (in your brain).

The perceptron tries to decide if you should go to a concert.

Is the artist good? Is the weather good?

What weights should these facts have?

Criteria	Input	Weight
Artists is Good	**x1** = 0 or 1	**w1** = 0.7
Weather is Good	**x2** = 0 or 1	**w2** = 0.6
Friend will Come	**x3** = 0 or 1	**w3** = 0.5
Food is Served	**x4** = 0 or 1	**w4** = 0.3

.**The Perceptron Algorithm**

Frank Rosenblatt suggested this algorithm:

1. Set a threshold value:

- Threshold = 1

2. Multiply all inputs with its weights:

- x1 * w1 = 1 * 0.7 = 0.7
- x2 * w2 = 0 * 0.6 = 0
- x3 * w3 = 1 * 0.5 = 0.5
- x4 * w4 = 0 * 0.3 = 0

3. Sum all the results:

- 0.7 + 0 + 0.5 + 0 = 1.2 (The Weighted Sum)

4. Activate the Output:

- Return true if the sum > 1 ("Yes I will go to the Concert")

Training a Perceptron

Training a perceptron involves adjusting the weights and the bias based on the perceptron's performance on training data. **The perceptron learning rule**, also known as **the delta rule**, updates the weights and bias according to the following rule:

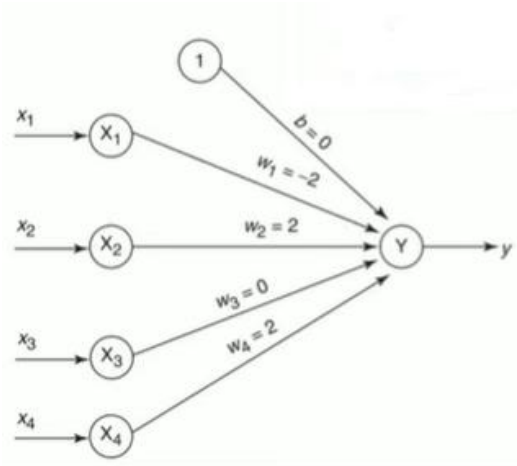
wi(new) = wi(old) + learning_rate * (expected_output - predicted_output) * xi

b(new) = b(old) + learning_rate * (expected_output - predicted_output)

where:

- **wi(new)** and **wi(old)** are the new and old weights, respectively,
- **learning_rate** is a small positive value that controls the magnitude of the weight update,
- **expected_output** is the true class label from the training data,
- **predicted_output** is the output produced by the perceptron,
- **xi** is the current input value.

The learning rate is a hyperparameter that determines how quickly the perceptron can adjust its weights. A small learning rate may lead to slow convergence, whereas a large learning rate may cause the perceptron to overshoot the optimal solution.



Input					Target
x_1	x_2	x_3	x_4	b	(t)
1	1	1	1	1	1
−1	1	−1	−1	1	1
1	1	1	−1	1	−1
1	−1	−1	1	1	−1

- Find the weights required to perform the following classification using perceptron network.
- The vectors (1, 1, 1, 1) and (- 1, 1 - 1, - 1) are belonging to the class 1, vectors (1, 1, 1, - 1) and (1, - 1, - 1, 1) are belonging to the class -1.
- Assume learning rate as 1
- and Initial weights as 0.

Input					Target
x_1	x_2	x_3	x_4	b	(t)
1	1	1	1	1	1
−1	1	−1	−1	1	1
1	1	1	−1	1	−1
1	−1	−1	1	1	−1

$$y_{in} = b + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4$$

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

Activation function

$$\Delta w_1 = \alpha t x_1;$$
$$\Delta w_2 = \alpha t x_2;$$
$$\Delta w_3 = \alpha t x_3;$$
$$\Delta w_4 = \alpha t x_4;$$
$$\Delta b = \alpha t$$

Where alpha is learning rate

't' is target

'x' is i/p

Change in weight = w1+delta_w1



$$y_{in} = b + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4$$

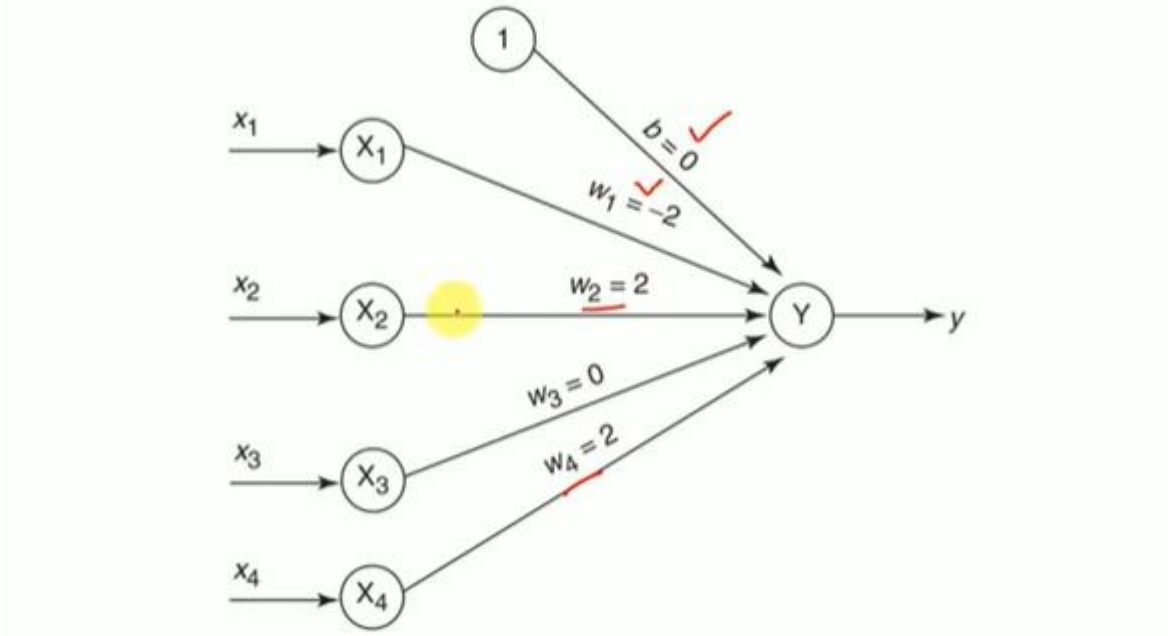
$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

Inputs				Target	Net input	output	Weight changes					Weights				
x_1	x_2	x_3	x_4	(t)	(y_{in})	(y)	(Δw_1	Δw_2	Δw_3	Δw_4	Δb)	w_1 (0	w_2 0	w_3 0	w_4 0	b 0)
EPOCH-1																
(1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1
(-1	1	-1	-1	1	-1	-1	-1	1	-1	-1	1	0	2	0	0	2
(1	1	1	-1	-1	4	1	-1	-1	-1	1	-1	-1	1	-1	1	1
(1	-1	-1	1	-1	1	1	-1	1	1	-1	-1	-2	2	0	0	0

$$\Delta w_1 = \alpha t x_1;$$
$$\Delta w_2 = \alpha t x_2;$$
$$\Delta w_3 = \alpha t x_3;$$
$$\Delta w_4 = \alpha t x_4;$$
$$\Delta b = \alpha t$$

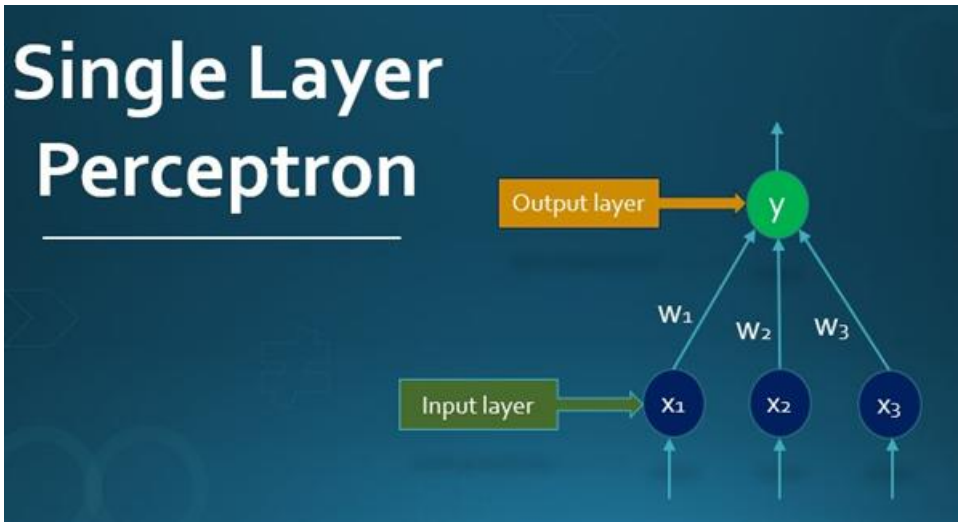
EPOCH-2																
(1	1	1	1	1	0	0	1	1	1	1	1	-1	3	1	1	1
(-1	1	-1	-1	1	3	1	0	0	0	0	0	-1	3	1	1	1
(1	1	1	-1	-1	4	1	-1	-1	-1	1	-1	-2	2	0	2	0
(1	-1	-1	1	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0

EPOCH-3																
(1	1	1	1	1	2	1	0	0	0	0	0	-2	2	0	2	0
(-1	1	-1	-1	1	2	1	0	0	0	0	0	-2	2	0	2	0
(1	1	1	-1	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0
(1	-1	-1	1	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0



Types of perceptron

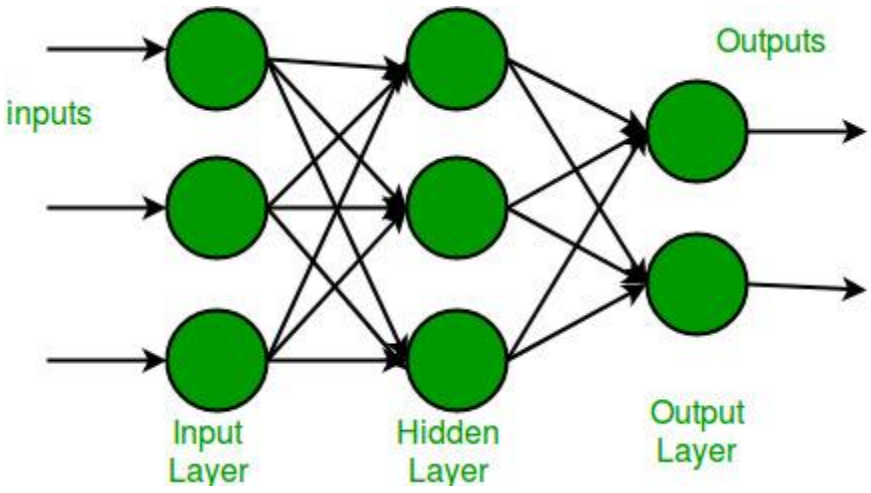
Single-Layer Perceptron is a type of perceptron is limited to learning linearly separable patterns. It is effective for tasks where the data can be divided into distinct categories through a straight line. While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear.



Multi-Layer Perceptron (MLP)

Multi-Layer Perceptron possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

- **Input Layer**: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers**: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer**: The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

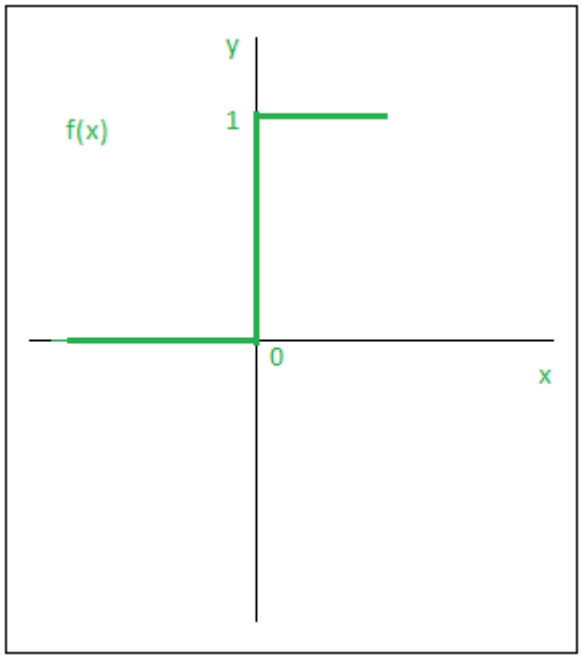


Every connection in the diagram is a representation of the fully connected nature of an MLP. This means that every node in one layer connects to every node in the next layer. As the data moves through the network each layer transforms it until the final output is generated in the output layer.

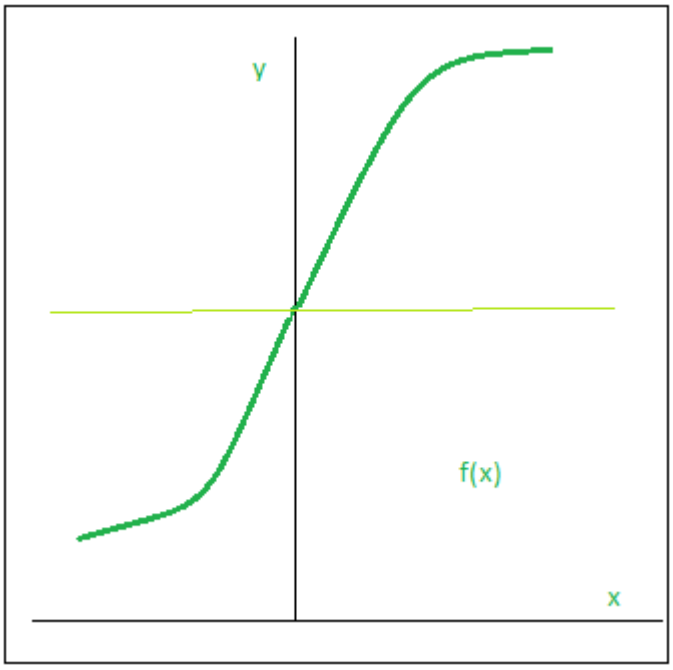
Types of Activation Functions –
Several different types of activation functions are used in Deep Learning. Some of them are explained below:

Step Function: Step Function is one of the simplest kind of activation functions. In this, we consider a threshold value and if the value of net input say **y** is greater than the threshold then the neuron is activated. Mathematically,

f(x)=1,if x>=0*f*(*x*)=1,if x>=0
f(x)=0,if x<0*f*(*x*)=0,if x<0



Sigmoid Function: Sigmoid function is a widely used activation function. It is defined as:
Formula: sigma(x) = 1 / (1 + exp(-x))
Where **x** is the weighted sum
Graphically,



It takes any input number and converts it into a value between 0 and 1.

For example:

- Input = **-10** → Output ≈ **0**
- Input = **0** → Output = **0.5**
- Input = **10** → Output ≈ **1**

Example: Pass or Fail

Suppose we want a neural network to predict whether a student will **PASS or FAIL**.
The neural network calculates some value:

$$x = 2$$

Sigmoid converts it:

$$2 \rightarrow 0.88$$

So:
0.88 = 88% chance of PASS
We can use a threshold of **0.5**:

Sigmoid Output Prediction

< 0.5	FAIL
≥ 0.5	PASS

Therefore:

$$0.88 > 0.5$$

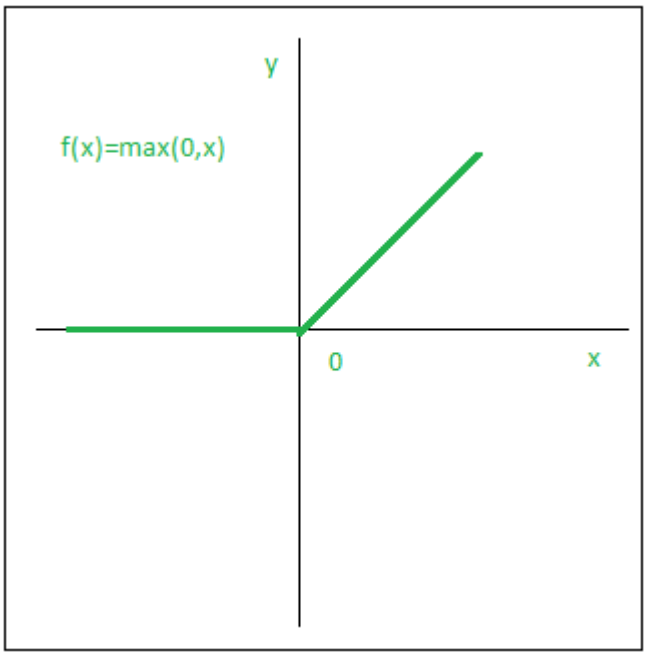
- **Prediction = PASS**

Note: Negative→0, Zero→0.5, positive →1

ReLU: The ReLU function is the Rectified linear unit. It is the most widely used activation function. It is defined as:

$f(x)=\max(0,x)$

Graphically,



The main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time.

If you look at the ReLU function if the input is negative it will convert it to zero and the neuron does not get activated.

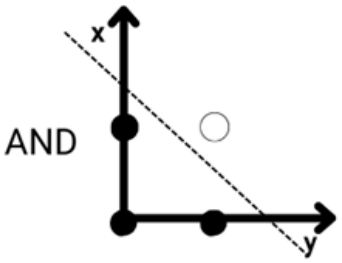
Computational time is less.

Linear Separability

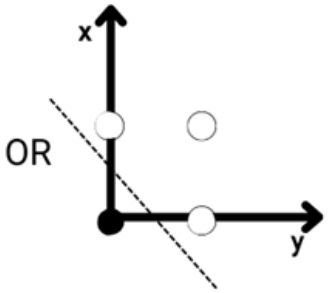
Perceptron Rule For Exclusive-OR Function

Logic gates are the basic building blocks of digital circuits. They decide which set of input signals will trigger the circuit using boolean operations. They commonly have two inputs in binary form (0,1) and produce one output (0 or 1) after performing a boolean operation.

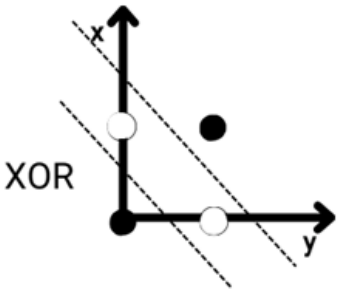
X1	X2	Y
F	F	0
F	T	0
T	F	0
T	T	1



X1	X2	Y
F	F	0
F	T	1
T	F	1
T	T	1



X1	X2	Y
F	F	0
F	T	1
T	F	1
T	T	0



A linear line can easily separate data points of OR and AND. So we can use a single-layer perceptron here.

Whereas, to separate data points of XOR, we need two linear lines or can add a new dimension and then separate them using a plane. Multi-layer Perceptron will work better in this case.

$$y = x_1\overline{x_2} + \overline{x_1}x_2$$

$$y = z_1 + z_2$$

where

$$z_1 = x_1\overline{x_2} \quad \text{(function 1)}$$

$$z_2 = \overline{x_1}x_2 \quad \text{(function 2)}$$

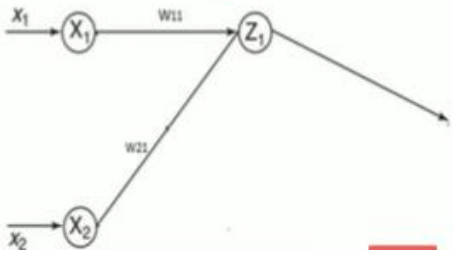
$$y = z_1 \text{ (OR) } z_2 \quad \text{(function 3)}$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

- First function $z_1 = x_1 \overline{x_2}$
- Assume the Initial weights are $w_{11} = w_{21} = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + 1 * 0 = 0 \ (out = 0)$
- $(0,1) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + 1 * 1 = 1 \ (out = 1)$
  - $w_{ij} = w_{ij} + n(t - o)x_i$
  - $w_{11} = 1 + 1.5 * (0 - 1) * 0 = 1$
  - $w_{21} = 1 + 1.5 * (0 - 1) * 1 = -0.5$

X_1	X_2	Z_1
0	0	0
0	1	0
1	0	1
1	1	0

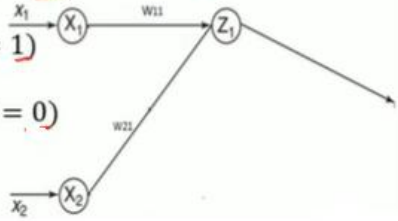
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



- First function $z_1 = x_1 \overline{x_2}$
- Assume the Initial weights are $w_{11} = 1 \ and \ w_{21} = -0.5$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + (-0.5) * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + (-0.5) * 1 = -0.5 \ (out = 0)$
- $(1,0) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 1 + (-0.5) * 0 = 1 \quad (out = 1)$
- $(1,1) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 1 + (-0.5) * 1 = 0.5 \quad (out = 0)$

X_1	X_2	Z_1
0	0	0
0	1	0
1	0	1
1	1	0

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



Now calculate Z2

- First function $z_2 = \overline{x_1} x_2$
- Assume the Initial weights are $w_{12} = w_{22} = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{2in} = w_{ij} * x_i = 1 * 0 + 1 * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow Z_{2in} = w_{ij} * x_i = 1 * 0 + 1 * 1 = 1 \quad (out = 1)$
- $(1,0) \rightarrow Z_{2in} = w_{ij} * x_i = 1 * 1 + 1 * 0 = 1 \quad (out = 1)$
  - $w_{ij} = w_{ij} + n(t - o)x_i$
  - $w_{12} = 1 + 1.5 * (0 - 1) * 1 = -0.5$
  - $w_{22} = 1 + 1.5 * (0 - 1) * 0 = 1$

X_1	X_2	Z_2
0	0	0
0	1	1
1	0	0
1	1	0

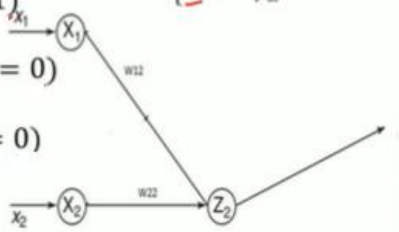
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



- First function $z_2 = \overline{x_1} x_2$
- Assume the Initial weights are $w_{12} = -0.5 \; and \; w_{22} = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 0 + 1 * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 0 + 1 * 1 = 1 \quad (out = 1)$
- $(1,0) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 1 + 1 * 0 = -0.5 \quad (out = 0)$
- $(1,1) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 1 + 1 * 1 = 0.5 \quad (out = 0)$

x_1	x_2	z_2
0	0	0
0	1	1
1	0	0
1	1	0

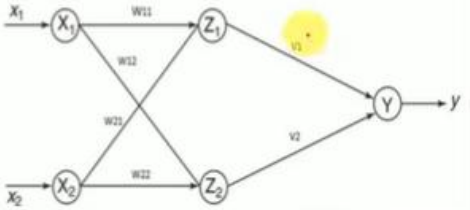
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} \leq \theta \end{cases}$$



First function $y = z_1 \; (\text{OR}) \; z_2 \qquad y_{in} = z_1 v_1 + z_2 v_2$

x_1	x_2	z_1	z_2	y
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

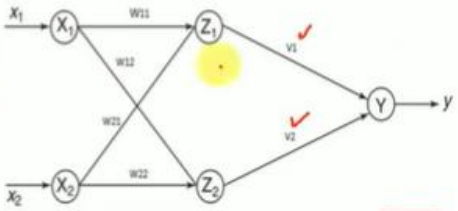
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} \leq \theta \end{cases}$$



- First function $y = z_1 \; (\text{OR}) \; z_2 \qquad y_{in} = z_1 v_1 + z_2 v_2$
- Assume the Initial weights are $v_1 = v_2 = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow y_{in} = v_i * x_i = 1 * 0 + 1 * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow y_{in} = v_i * x_i = 1 * 0 + 1 * 1 = 1 \quad (out = 1)$
- $(1,0) \rightarrow y_{in} = v_i * x_i = 1 * 1 + 1 * 0 = 1 \quad (out = 1)$
- $(0,0) \rightarrow y_{in} = v_i * x_i = 1 * 0 + 1 * 0 = 0 \quad (out = 0)$

x_1	x_2	z_1	z_2	y
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} \leq \theta \end{cases}$$



Final network

